DBC 1200

Onboard Inertial Sensor Calibration Procedure

Customer Technical Document

Rev 1.1

BWI Group Proprietary

DO NOT DISTRIBUTE OR COPY WITHOUT BWI APPROVAL

Revision History

Doc Version	Date	Change Summary
1.0	5/29/15	Initial Draft for Calibration Procedure CTD.
1.1	6/11/15	Changes made to the text of the document for added clarity.

Tables of Contents

Revision History....2
Tables of Contents....3
1. Reference Documents....4
2. Introduction to Calibration or Trim Process....4
2.1 EBCM Inertial Sensor Trim Information....4
DID $3027: Inertial Sensor Trim Information....4
3. Diagnostic Services for Trim Process....5
3.1 SecurityAccess Service (SId $27)....5
3.1.1 SId $27 - Request Message Definition....7
3.1.2 SId $27 - Sub-function Parameter Level Definition....7
3.1.3 SId $27 - Data parameter Definition....7
3.1.4 SId $27 - Request Message Data Bytes....7
3.1.5 SId $27 - Positive Response Message....8
3.1.6 SId $27 - Negative Response Message....8
3.2 ReadDataByIdentifier Service (SId $22)....9
3.2.1 SId $22 - Request Message Definition....9
3.2.2 SId $22 - Sub-function Parameter Level Definition....9
3.2.3 SId $22 - Data Parameter Definition....10
3.2.4 SId $22 - Positive Response Message Data Bytes....10
3.2.5 SId $22 - Negative Response Message Data Bytes....10
3.3 RoutineControl service (Sid $31)....11
3.3.1 Request Message definition....11
3.3.2 Sub-function Parameter Level definition....11
3.3.3 Data parameter definition....11
3.3.4 Request Message Data Bytes....12
3.3.5 Positive Response Message....13
3.3.6 Negative Response Message....14
4. OE Assembly Plant Process....15
4.1 Vehicle requirements for trimming the sensor:-....15
4.2 Steps for trimming the sensor once the vehicle requirements in section 4.1 are met:-....15
4.3 OE Assembly Plant Process Flow Chart....16

1. Reference Documents

- DBC 1200 Pressure-Build Electro-Hydraulic Control Unit ECU Diagnostic Communication for ISO 14229 Customer Technical Document Specification # 18116030 v 1.9 5-13-2015.
- DBC1200 Security Algorithm documentation for seed and key access with Service $27

2. Introduction to Calibration or Trim Process

DBC 1200 has an onboard Inertial Sensor (combination or combo sensor) which computes lateral acceleration, longitudinal acceleration and yaw rate. These 3 components are used by the safety features such as ABS, ESC, TCS, HSA, etc to perform their functionality. For correct computation of these 3 components it is necessary to calibrate or "trim" the sensor to a "0" value while in the vehicle using the trim process mentioned in this document. Trim process can be done by the customer at the vehicle assembly plant. The main advantage of trimming the controller in the vehicle is that some of the error in the mount bracket can be taken into account. If the error is too large the trim process will be terminated and a NEC will be sent to the test tool.

2.1 EBCM Inertial Sensor Trim Information

DID $3027: Inertial Sensor Trim Information

Using DID $3027 the OEM can read inertial sensor trim information as outlined below:

$3027 EBCM Inertial Sensor Trim Information						
Data Byte #	Bit #	Description	Logic	Units	Range	Reporting Resolution
1	all	Process status	Inertial Sensor Trim Status	NA	NA	Bit field
		Bitfield Values	All Bits Fault Byte Bit 7 Successful Average Complete Bit 6 Exceeded Total Trim Adjust Time Bit 5 Excessive Jerk Encountered Bit 4 Exceeded Accel Span Bit 3 Exceeded Trim Temperature Range Bit 2 Accel Offset Exceeds Allowable Max Bit 1 Abnormal Sensor Status Encountered Bit 0 Successful Trim Complete			
2	all	Axis y Zeroing Value	ECU Y Accel Offset (In reference to the Inertial sensor)	g	-0.25 to 0.25	1/512
3	all	Axis z Zeroing Value	ECU Z Accel Offset (In reference to the Inertial sensor)	g	-0.25 to 0.25	1/512

Table: EBCM Inertial Sensor Trim Information

Where,

Byte 1 Inertial Sensor trim bits:

Bit 7 – Successful average complete – An average of ECU_Y_Axis_Accel (lateral acceleration) and ECU_Z_Axis_Accel (Longitudinal acceleration) is computed successfully to get a clean signal.

Bit 6 – Exceeded Total Trim Adjust Time – If the total time taken for averaging in bit 7 exceeds inertial sensor trim duration mentioned $31 then this flag is set.

Bit 5 – Excessive Jerk Encountered – If a jerk is encountered while trimming the sensor which exceeds 0.01404 g's/loop then this flag is set.

Bit 4 – Exceeded Accel Span – If this value exceeds 0.07996 g's then this flag is set.

Bit 3 – Exceeded Trim Temperature Range – If the temperature is less than 4 degrees Celsius or if it exceeds 61 degree Celsius then this flag is set. This flag is not set when the temperature is between 4 – 61 degree Celsius.

Bit 2 – Accel Offset Exceeds Allowable Max – If this value exceeds 0.05005 g's then this flag is set.

Bit 1 – Abnormal Sensor Status Encountered – If any of the above bits are set this bit sets to flag the fault.

Bit 0 – Successful Trim Complete – On successfully completing the trim process this flag is set.

Byte 2 ECU Y Accel Offset – This is lateral acceleration offset in reference to the inertial sensor calculated during the trim process. For e.g., a value of 0.10 g means that the lateral acceleration has an offset of 0.10 g's.

Byte 3 ECU Z Accel Offset - This is longitudinal accelerometer offset in reference to the inertial sensor calculated during the trim process. For e.g., a value of 0.10 g means that the longitudinal acceleration has an offset of 0.10 g's.

Note: The numbers mentioned above are typical calibration values.

3. Diagnostic Services for Trim Process

The following services are used to trim the Inertial Sensor. Diagnostic services $19, $22, $27 and $31 are required to complete the trim process. Service $27, $31 and $22 are covered below. Note: Reference 1 can be used for a more in depth look at the 14229 Diagnostic services.

3.1 SecurityAccess Service (SId $27)

For more complete details see Reference 1 - DBC 1200 Pressure-Build Electro-Hydraulic Control Unit ECU DiagnosticCommunication for ISO 14229 Customer Technical Document Specification # 18116030 v 1.9 5-13-2015

The purpose of this service is to provide a means to access data and/or diagnostic services, which have restricted access for security, emissions, or safety reasons.

Note: The security service $27 uses a proprietary security algorithm. This service could protect or not allow access, for example, to ECU reprogramming and DID writes. Security algorithm details is available upon request.

A typical example of the use of this service is as follows:

- Client requests the "Seed"
- Server sends the "Seed"

- Client sends the “Key” (appropriate for the Seed received)
- Server responds that the “Key” was valid and that it will unlock itself

The client shall request the server to "unlock" by sending the service SecurityAccess ‘requestSeed’ message. The server shall respond by sending a "seed" using the service SecurityAccess ‘requestSeed’ positive response message. The client shall then respond by returning a "key" number back to the server using the appropriate service SecurityAccess ‘sendKey’ request message. The server shall compare this "key" to one internally stored. If the two numbers match, then the server shall enable ("unlock") the client's access to specific services/data and indicate that with the service SecurityAccess ‘sendKey’ positive response message. If the two numbers do not match, this shall be considered a false access attempt. If access is rejected for any other reason, it shall not be considered a false access attempt. An invalid key requires the client to start over from the beginning with a SecurityAccess 'requestSeed' message.

If a server is already unlocked when a SecurityAccess ‘requestSeed’ message is received, that server shall respond with a SecurityAccess ‘requestSeed’ positive response message service with a seed value equal to zero (0). The client shall use this method to determine if a server is locked by checking for a non-zero seed.

There shall always be a fixed relationship for each level of security supported so that the sendKey sub-function parameter value used for any given security level shall be equal to the requestSeed sub-function parameter value used for that security level plus one.

Only one security level shall be active at any instant of time. For example, if the security level associated with requestSeed 03 hex is active and a tester request is successful in unlocking the security level associated with requestSeed 01 hex, then only the secured functionality supported by the security level associated with requestSeed 01 hex shall be unlocked at that time. Any additional secured functionality that was previously unlocked by the security level associated with requestSeed 03 hex shall no longer be active. The security levels numbering is arbitrary and does not imply any relationship between the levels.

Attempts to access security shall not prevent normal vehicle communications or other diagnostic communication.

Servers, which provide security shall support reject messages if a secure service is requested while the server is locked.

Some diagnostic functions/services requested during a specific diagnostic session may require a successful security access sequence. In such case the following sequence of services shall be required:

- DiagnosticSessionControl service
- SecurityAccess service
- Secured diagnostic service

There are different accessModes allowed for an enabled diagnosticSession (session started) in the server.

Please note that some customer applications will only use one set of seed and key pair for both DID write and reprogramming securities. Others will use each set for one single purpose.

Added note: Only two byte seed/key supported in this revision of the Inertial Sensor Trim process.

This service is only supported if requested by in Physical addressing mode.

3.1.1 SId $27 - Request Message Definition

The elements of the request message are described in the following paragraphs.

3.1.2 SId $27 - Sub-function Parameter Level Definition

The sub-function parameter securityAccessType indicates to the server the step in the process for this service, the level of security the client wants to access and the format of the seed and key.

Examples:

- "requestSeed=01 hex" identifies a fixed relationship between "requestSeed=01 hex" and "sendKey=02 hex"
- "requestSeed=03 hex" identifies a fixed relationship between "requestSeed=03 hex" and "sendKey=04 hex"

Values are defined in 2.3.1-1 for requestSeed and sendKey (suppressPosRspMsgIndicationBit (bit 7) not shown). Specific for Inertial Sensor Trim Process.

Hex (bit 6-0)	Description	Cvt	Mnemonic
01	requestSeed Request Inertial Sensor Trim Process Seed	U	RSD
02	Send Key Send Inertial Sensor Trim Process Key	U	SK

Table - SId $27 Request message sub-function parameter definition

3.1.3 SId $27 - Data parameter Definition

This service does not support data parameters in the request message.

3.1.4 SId $27 - Request Message Data Bytes

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	SecurityAcces Request Service Id	M	27	SA
#2	sub-function = [securityAccessType = requestSeed]	M	01	LEV_ SAT_RSD

Table– SId $27 Request message definition - sub-function = requestSeed

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	SecurityAcces Request Service Id	M	27	SA
#2	sub-function = [securityAccessType = sendKey]	M	 02	LEV_ SAT_SK
 #3 #4	securityKey[] = [key#1 (high byte) key#2 (low byte)]	 M M	 00-FF 00-FF	SECKEY_ KEYHB KEYLB

Table– SId $27 Request message definition - sub-function = sendKey

3.1.5 SId $27 - Positive Response Message

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	SecurityAccess Response Service Id	S	67	SAPR
#2	securityAccessType	M	01	SAT_
 #3 #4	securitySeed[] = [seed#1 (highbyte) seed#m (low byte)]	 C C	 00-FF 00-FF	SECSEED_ SEED1HB SEED2LB

Table - SId $27 requestSeed Positive Response Message

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	SecurityAccess Response Service Id	S	67	SAPR
#2	securityAccessType	M	02	SAT_

Table - SId $27 sendKey Positive Response Message

3.1.6 SId $27 - Negative Response Message

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	negativeResponse Service Id	S	7F	NR
#2	testerPresent Request Service Id	M	27	TP
#3	ResponseCode = [subFunctionNotSupported incorrectMessageLengthOrInvalidFormat ConditionsNotCorrect requestSequenceError invalidKey exceededNumberOfAttempts requiredTimeDelayNotExpired serviceNotSupportedInActiveSession]	M	 12 13 22 24 35 36 37 7F	RC SFNS IMLOIF CNC RSE IK ENOA RTDNE SNSIAS

Table - SId $27 Negative Response Message

Response Code (hex)	Condition
12	subFunctionNotSupported Requested sub-function is not supported
13	incorrectMessageLengthOrInvalidFormat Wrong length for the specific request (Note: BWI can support configuring software to check for only length shorter than specified)
22	ConditionsNotCorrect Request is functional addressed
24	requestSequenceError Send if the 'sendKey' sub-function is received without first receiving a 'requestSeed' request message.
35	invalidKey Send if an expected 'sendKey' sub-function value is received and the value of the key does not match the server's internally stored/calculated key.
36	exceededNumberOfAttempts Send if the delay timer is active due to exceeding the maximum number of allowed false access attempts.
37	requiredTimeDelayNotExpired Send if the delay timer is active and a request is transmitted.
7F	serviceNotSupportedInActiveSession The service is not enabled in the current diagnostic session.

Table - SId $27 Response Code

3.2 ReadDataByIdentifier Service (SId $22)

For more complete details see Reference 1 - DBC 1200 Pressure-Build Electro-Hydraulic Control Unit ECU DiagnosticCommunication for ISO 14229 Customer Technical Document Specification # 18116030 v 1.9 5-13-2015.

The ReadDataByIdentifier service allows the client to request data record values from the server identified by one or more dataIdentifiers.

The server limits the number of dataIdentifiers that can be simultaneously requested as 1.

Upon receiving a ReadDataByIdentifier request, the server will access the data elements of the records specified by the dataIdentifier parameter(s) and transmit their value in one single ReadDataByIdentifier positive response containing the associated dataRecord parameter(s).

3.2.1 SId $22 - Request Message Definition

The elements of the request message are described in the following paragraphs.

3.2.2 SId $22 - Sub-function Parameter Level Definition

This service does not use a sub-function parameter.

3.2.3 SId $22 - Data Parameter Definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	ReadDataByIdentifier Request Service Id	M	22	RDBLI
#2 #3	dataIdentifier = [byte#1 (MSB) byte#2]	M	 00-FF 00-FF	DID HB LB

Table - ReadDataByIdentifier Request Message

3.2.4 SId $22 - Positive Response Message Data Bytes

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	**readDataByIdentifier Request Service Id**	**S**	**62**	**RDBI**
#2 #3	dataIdentifier =[byte#1 (MSB) byte#2]	M	 00-FF 00-FF	DID HB LB
#4 : #n	datarecord#1 = [data#1 : data#k	 M : C	 xx, : xx	DREC_ DATA_1 : DATA_k

Table - ReadDataByIdentifier Positive Response Message

3.2.5 SId $22 - Negative Response Message Data Bytes

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	**negativeResponse Service Id**	**S**	**7F**	**NR**
#2	**readDataByIdentifier Request Service Id**	**M**	**22**	**REI**
#3	ResponseCode = [incorrectMessageLengthOrInvalidFormat requestOutOfRange]	M	 13 31	 IMLOIF ROOR

Table - ReadDataByIdentifier Negative Response Message

Hex	Conditions
13	incorrectMessageLengthOrInvalidFormat Wrong length for the specific request
31	RequestOutOfRange This response code will be sent if: Any DataIdentifier in the request is invalid (reserved or not defined).

Table - Diagnostic Services $22 Response Code

3.3 RoutineControl service (Sid $31)

For more complete details see Reference 1 - DBC 1200 Pressure-Build Electro-Hydraulic Control Unit ECU DiagnosticCommunication for ISO 14229 Customer Technical Document Specification # 18116030 v 1.9 5-13-2015

IMPORTANT — The server and the client shall meet the request and response message behaviour as specified in section **Error! Reference source not found. Error! Reference source not found.** in the event that those addressing methods are implemented for this service.

Service	Description
RoutineControl	The client requests to start stop a routine in the server(s) or requests the routine results.

Table: — Remote Activation of Routine functional unit

3.3.1 Request Message definition

3.3.2 Sub-function Parameter Level definition

The sub-function parameters are used by this service to select the control of the routine. Explanations and usage of the supported levels are detailed below (suppressPosRspMsgIndicationBit (bit 7) not shown).

A_Data byte	Description	Cvt	Mnemonic
01	startRoutine This parameter specifies that the server shall start the routine specified by the routineIdentifier.	U	STR

Table: Request message sub function definition

3.3.3 Data parameter definition

The following data-parameters are defined for this service:

Definition
routineIdentifier This parameter identifies a server local routine and is out of the range of defined dataIdentifiers (see ISO 14229 Annex F.1).
routineControlOptionRecord This parameter record contains either — Routine related Parameters.* (Added to BWI specific usage)

Table: — Request message data parameter definition

3.3.4 Request Message Data Bytes

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	RoutineControl Request Service Id	M	31	RC
#2	sub-function = [routineControlType]	M	01	LEV_ RCTP_
#3 #4	routineIdentifier [] = [byte#1 (MSB) byte#2]	M M	00-FF 00-FF	RI_ B1 B2
#5 : #n	routineControlOptionRecord [] = routineControlOption #1 routineControlOption #m]	U : U	00-FF : 00-FF	RSR_ RS_ : RS _
C: This parameter is user optional to be present for sub-function parameter startRoutine and stopRoutine.				

Table: RoutineControl request message

RequestParameter Name	S/W Variable Name/Value	Display/Resolution
routineIdentifier [] = [		
byte#1 (MSB)	F0	Hex
byte#2]	00	Hex
routineControlOptionRecord [] = [		
routineControlOption #1	Inertial Sensor Trim Samples	1= 10 Samples (takes100ms)
routineControlOption #2]	Inertial Sensor Trim Duration	1= 20 ms

Table: RoutineControl $F000 Request Parameter Definition*

3.3.5 Positive Response Message

A_Data byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1	RoutineControl Response Service Id	S	71	RCPR
#2	routineControlType	M	01	RCTP_
#3 #4	routineIdentifier [] = [byte#1 (MSB) byte#2]	M M	00-FF 00-FF	RI_ B1 B2
#5 : #n	routineStatusRecord[] = [routineStatus#1 : routineStatus#m]	U : U	00-FF : 00-FF	RSR_ RS_ : RS _

Table: — Positive response message definition

RequestParameter Name	S/W Variable Name/Value	Parameter code
routineIdentifier [] = [		
byte#1 (MSB)	F0	Hex
byte#2]	00	Hex
routineStatusRecord[] = [		
routineStatus byte #1	Inertial Sensor Trim Status	Bit field
routineStatus byte #2	ECU Y Accel Offset	1/512
routineStatus byte #3]	ECU X Accel Offset	1/512

Table: RoutineControl $F000 Response Parameter Definition*

3.3.6 Negative Response Message

A_PDU parameter	Parameter Name	Cvt	Hex Value	Mnemonic
#1	**negativeResponse Service Id**	**S**	**7F**	**NR**
#2	**writeDataByIdentifier Request Service Id**	**M**	**2E**	**WDBLI**
#3	ResponseCode = [generalReject subFunctionNotSupported incorrectMessageLengthOrInvalidFormat ConditionsNotCorrect requestOutOfRange securityAccessDenied reqCorrectlyRcvd-RspPending serviceNotSupportedInActiveSession VehicleSpeedLimitExceeded]	M	10 12 13 22 31 33 78 7F 88	GR SFNS IMLOIF CNC ROOR SAD RCR-RP SNSIAS VSLE

Table: — Supported negative response codes

4. OE Assembly Plant Process

The DBC1200 EHCU will be shipped from the BWI Production Plant with a fully trimmed or calibrated onboard inertial sensor. Controlled brake functions will be active including ABS, TCS, and ESP. At this point the onboard inertial sensor can be trimmed again in the vehicle if necessary at the OEM plant to compensate for error introduced from mounting bracket. Vehicle requirements for trimming the sensor should be met before trim operation – see section 4.1. Once vehicle is stable, the steps needed to be performed in order to trim the sensor can be executed – see below.

4.1 Vehicle requirements for trimming the sensor:-

1. Vehicle should be on a levelled surface.
2. Vehicle should be stationary.
3. Engine should not be running.
4. Please do not open, close or slam the doors while trimming the sensor.

4.2 Steps for trimming the sensor once the vehicle requirements in section 4.1 are met:-

1. Initiate diagnostic session using “$10 03” ExtendedDiagnosticSession
2. Unlock the ECU using SId $27 as mentioned in this document.
3. Trim the sensor using SId $31 i.e., “$31 01 F0 00 XX YY” where XX is the number of samples which is typically 10 ($0A) and YY is the duration which is typically 200 ms ($C8).
4. ECU may respond with a “$7F 78” during the Trim process which means the trim is occurring and a diagnostic response is pending. When the trim process has completed either successful or not successful, the ECU then responds with a positive response or negative response.
5. On successfully trimming the sensor a positive response of “$71 01 F0 00 XX YY ZZ” is received where XX is the pass/fail status, YY is Y-axis offset and ZZ is the Z- axis offset in reference to the inertial sensor chip.
6. Using SId $22, the “DID $3027 can be read to determine inertial sensor trim information”. This will check the status of the trim process and whether the part trim was successful. If the trim procedure failed, Byte 1 of DID $3027 can be used to determine why the trim process has failed.
7. SId $19 to read DTC information is used to check if diagnostic trouble codes “DTC 55: ECU Internal Fault” and “ECU sub fault code bit 28: Integral Inertial Sensor Fault” have been cleared.

Notes:-

1. ECU must be in normal mode which is typically > ~500ms after the ECU wakes up.
2. Wheel speeds must be 0 to perform the trim process.
3. Tester may choose to try again if a negative or no response is received before the ECU diagnostic session times out at ~5.5 seconds.
4. Typically, the trim procedure will complete within a 5 second period. However, during trim procedure development it may be necessary for the tester to send a $3E tester present message to prevent diagnostic session timeout.

DO NOT DISTRIBUTE OR COPY WITHOUT BWI APPROVAL

4.3 OE Assembly Plant Process Flow Chart

The DBC1200 EHCU shipped from the BWI Production Plant with trimmed inertial sensor.

Initiate diagnostic session using "10 03" and Unlock the ECU using SId $27

Enter extended Diagnostic session $10 subfunction 04:

712 02 10 03 00 00 00 00 00
702 06 50 03 00 00 00 00 00

Enter Security $27 for DID Write – subfunction

712 02 27 01 00 00 00 00 00
702 04 67 01 12 34 00 00 00

712 04 27 02 AA BB 00 00 00
702 02 67 02 00 00 00 00 00

Session control

Security Access

Trim the sensor using SId $31 i.e., "31 01 F0 00 XX YY"

ECU responds with "7F 78" within 50 ms which means response pending. ECU then responds with a positive response or negative response when trim is complete.

Trim the sensor using SId $31:

712 06 31 01 F0 00 10 C8 00

Response Pending:

702 03 7F 31 78 00 00 00 00

Positive response:

702 07 71 01 F0 00 XX YY ZZ